

COMPTE RENDU

SDN & Network Softwarization — TP0 — Commandes de base réseau &
système

4e année Cybersécurité — École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
Filière : Cybersécurité
Année : 2025/2026
Enseignante : M. BOUHORMA
Date : 16 septembre 2026

Table des matières

Cadre du TP	3
1 Identifier le système	4
1.1 Identité du système	4
1.2 CPU et mémoire	4
2 Interfaces et adressage IP	5
2.1 Interfaces	5
2.2 Adresses IP	6
2.3 Loopback	7
3 Routage et chemin de sortie	8
3.1 Table de routage	8
3.2 Quelle route pour une destination ?	9
4 Tester la connectivité par couches	10
4.1 Mini-diagnostic	11
5 DNS : résoudre les noms	12
6 Voisinage local et chemin réseau	15
6.1 Voisins du LAN	15
6.2 Tracer le chemin	16
7 Ports, services et processus	17
7.1 Ports en écoute	17
7.2 Installer un vrai service Web	18
7.3 Tester depuis plusieurs points de vue	19
7.4 Processus et service	20
8 Logs et ressources système	22
9 Challenge final : la panne mystérieuse	24
9.1 Méthode obligatoire	24
9.2 Fiche d'incident à compléter	25

10 Synthèse et transition vers Network Virtualization	26
10.1 Commandes indispensables	26
10.2 Questions finales	26
10.2.1 Quelle différence entre <code>ip addr</code> et <code>ip route</code> ?	26
10.2.2 Pourquoi 8.8.8.8 peut-il répondre alors que google.com échoue ?	26
10.2.3 À quoi sert la passerelle par défaut ?	27
10.2.4 Quelle commande permet de savoir quel processus écoute sur TCP/80 ?	27
10.2.5 Quelle différence entre un processus et un service Linux ?	27
10.2.6 Pourquoi <code>ip route get</code> devient-il important avec plusieurs inter- faces réseau ?	27
Livrable étudiant	27

Cadre du TP

Niveau	Durée	Plateforme
4e CY & CCV	1 h 30	Ubuntu ou Kali Linux
Mode	Approche	Livrable
Individuel	Diagnostic réel	Compte rendu court

TABLE 1 – Cadre général du TP

Mission : Vous rejoignez une équipe réseau/sécurité/cloud. On vous fournit une machine Linux et on vous indique seulement : « la machine est connectée au réseau, mais certains services ne fonctionnent pas correctement ». Vous devez reconstruire son état uniquement depuis le terminal, localiser une panne et justifier votre diagnostic.

Objectif final :

**Système → Interface → IP → Route → Passerelle → Internet → DNS → Port
→ Service → Application**

Règle de travail :

- Ne recopiez pas seulement la sortie : interprétez-la ;
- À chaque test, notez ce que vous cherchez à vérifier ;
- En cas d'échec, descendez ou remontez d'un niveau dans la chaîne de diagnostic ;
- Les captures utiles seront demandées dans le livrable final.

Listing 1 – Commandes à maîtriser durant le TP

```
1 hostname    uname -a    whoami     lscpu      free -h
2 ip link     ip addr    ip route   ip route get ip neigh
3 ping        traceroute resolvectl dig      ss -lntp
4 curl        ps aux     top        systemctl  journalctl lsof      df -h     du
             -sh
```

Note sur l'environnement. Les relevés de ce compte rendu ont été effectués sur deux réseaux distincts : une partie sur le réseau Wi-Fi du campus (UIR, plage 10.85.0.0/16) et une partie sur mon réseau personnel à domicile (plage 192.168.11.0/24). Les adresses IP, passerelles et interfaces relevées diffèrent donc d'une section à l'autre selon le réseau utilisé au moment de la capture.

1 Identifier le système

Qui suis-je ? Sur quelle machine ? Avec quelles ressources ?

1.1 Identité du système

Listing 2 – Identité du système

```
1 whoami
2 hostname
3 uname -a
4 cat /etc/os-release
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ whoami
hathouti
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ hostname
Taha-inspiron-16
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ uname -a
Linux Taha-inspiron-16 7.0.0-28-generic #28~24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Wed Jul 1 15:50:57 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.4 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$
```

FIGURE 1 – Sortie des commandes d'identification du système

Élément	Valeur relevée
Utilisateur	hathouti
Hostname	Taha-inspiron-16
Distribution / version	Ubuntu 24.04.4 LTS (Noble Numbat)
Kernel Linux	7.0.0-28-generic
Architecture	x86_64

TABLE 2 – Relevé de l'identité du système

1.2 CPU et mémoire

Listing 3 – CPU et mémoire

```
1 lscpu
2 free -h
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ free -h
              total        utilisé        libre        partagé tamper/cache    disponible
Mem:          31Gi         17Gi         2,3Gi         4,4Gi          16Gi         13Gi
Échange:       4,0Gi           0B         4,0Gi
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$
```

FIGURE 2 – Sortie de lscpu et free -h

Élément	Valeur
CPU / vCPU	16 threads (8 cœurs physiques × 2 threads) – Intel Core i7-11800H @ 2.30 GHz
RAM totale	31 Gio
RAM disponible	13 Gio

TABLE 3 – Relevé CPU et mémoire

Question. *Pourquoi CPU et RAM deviennent-ils des ressources essentielles lorsqu'on virtualise plusieurs machines ou fonctions réseau ?*

Quand on virtualise, chaque machine virtuelle ou fonction réseau tourne comme un ensemble de processus sur l'hôte et se partage le **même** CPU physique et la **même** RAM. Ces deux ressources sont finies et directement mutualisées :

- **CPU** : chaque VM/VNF consomme des cœurs (ou des fractions de cœurs) pour traiter les paquets. Les threads de l'hôte (par exemple les 16 threads d'un i7-11800H) doivent être répartis entre l'hyperviseur et toutes les VM ; au-delà, on tombe en *sur-engagement*, l'ordonnanceur partage le temps CPU et la latence de traitement augmente ;
- **RAM** : chaque VM réserve sa propre mémoire (OS invité, tables de routage, buffers réseau). La RAM totale de l'hôte est un plafond strict ; une fois saturée, le système passe par le *swap* disque, bien plus lent, ce qui dégrade fortement les performances réseau ;
- En **VNF**, où des équipements matériels (routeurs, pare-feux) sont remplacés par du logiciel sur serveur générique, le débit et la latence dépendent donc directement du CPU et de la RAM disponibles : ce sont eux qui fixent le nombre de fonctions hébergeables et leur capacité de traitement.

2 Interfaces et adressage IP

Avant de parler réseau, identifiez exactement les interfaces disponibles.

2.1 Interfaces

Listing 4 – Lister les interfaces réseau

```
1 ip link
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: wlp0s20f3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DORMANT group default qlen 1000
   link/ether 60:a5:e2:64:75:d0 brd ff:ff:ff:ff:ff:ff
3: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN mode DEFAULT group default qlen 1000
   link/ether 52:54:00:9e:3b:49 brd ff:ff:ff:ff:ff:ff
5: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN mode DEFAULT group default
   link/ether 46:69:e6:25:eb:62 brd ff:ff:ff:ff:ff:ff
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$
```

FIGURE 3 – Sortie de `ip link`

Interface	État	Rôle supposé
lo	UP (state UNKNOWN)	Boucle locale (loopback)
wlp0s20f3	UP (LOWER_UP)	Carte Wi-Fi — interface active connectée au réseau
virbr0	DOWN (NO-CARRIER)	Pont virtuel libvirt/KVM — réseau des VM
docker0	DOWN (NO-CARRIER)	Pont virtuel Docker — réseau des conteneurs

TABLE 4 – Relevé des interfaces

À retenir : `lo` est l'interface loopback. Les interfaces Ethernet modernes portent souvent des noms comme `ens33`, `enp0s3` ou `eth0`.

2.2 Adresses IP

Listing 5 – Afficher les adresses IP

```
1 ip addr
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: wlp0s20f3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 60:a5:e2:64:75:d0 brd ff:ff:ff:ff:ff:ff
    inet 10.85.100.55/16 brd 10.85.255.255 scope global dynamic noprefixroute wlp0s20f3
        valid_lft 2242158sec preferred_lft 2242158sec
    inet6 fe80::f7dd:c516:6783:a940/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
    link/ether 52:54:00:9e:3b:49 brd ff:ff:ff:ff:ff:ff
    inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
        valid_lft forever preferred_lft forever
5: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 46:69:e6:25:eb:62 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$
```

FIGURE 4 – Sortie de `ip addr`

Élément	Valeur
Interface principale	wlp0s20f3 (Wi-Fi)
Adresse IPv4	10.85.100.55/16
Préfixe / CIDR	/16 (masque 255.255.0.0)
Adresse MAC	60 :a5 :e2 :64 :75 :d0

TABLE 5 – Relevé de l'adressage IP

Exercice. Si votre adresse est `192.168.1.35/24` : quel est le masque ? quel est le réseau ?

Le préfixe /24 signifie que les 24 premiers bits sont réservés au netID, ce qui laisse 8 bits pour les hosts.

Masque :

— En binaire : 1111 1111.1111 1111.1111 1111.0000 0000 ;

— En décimal : 255.255.255.0 ;

IP de l'hôte :

— En décimal : 192.168.1.35 ;

— En binaire : 1100 0000.1010 1000.0000 0001.0010 0011 ;

On annule tous les bits correspondant au Host ID pour obtenir l'adresse du réseau :

En binaire : 1100 0000.1010 1000.0000 0001.**0000 0000** ;

En décimal : **192.168.1.0/24** ;

Et en remplaçant tous les bits du Host ID par des 1 on obtient l'adresse de broadcast :

En binaire : 1100 0000.1010 1000.0000 0001.**1111 1111** ;

En décimal : **192.168.1.255** ;

Réseau : 192.168.1.0/24

— **Adresse du réseau** : 192.168.1.0 ;

— **Première adresse** : 192.168.1.1 ;

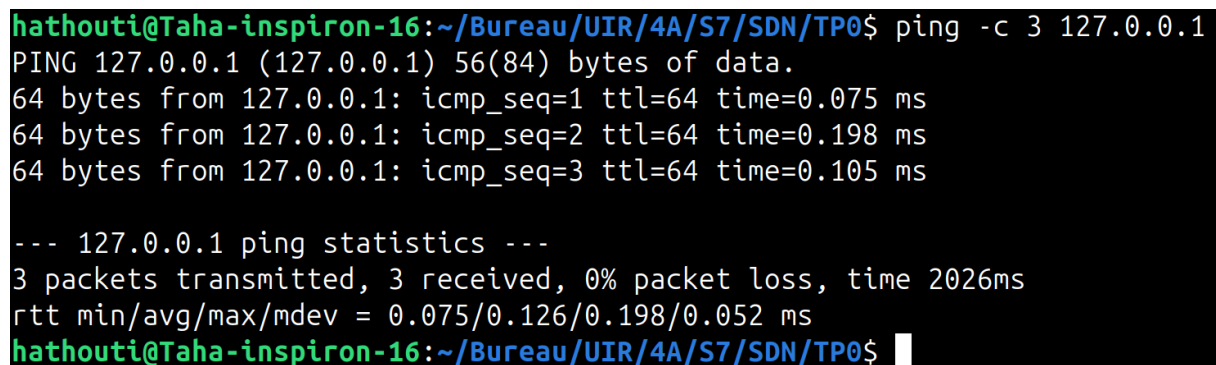
— **Dernière adresse** : 192.168.1.254 ;

— **Broadcast** : 192.168.1.255 ;

2.3 Loopback

Listing 6 – Test du loopback

```
1 ping -c 3 127.0.0.1
```



```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ ping -c 3 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.075 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.198 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.105 ms

--- 127.0.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2026ms
rtt min/avg/max/mdev = 0.075/0.126/0.198/0.052 ms
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$
```

FIGURE 5 – Sortie de ping -c 3 127.0.0.1

☒ Le test fonctionne

☐ Le test échoue

Question. *Expliquez pourquoi ce test peut fonctionner même sans connexion au LAN ou à Internet.*

L'adresse 127.0.0.1 est l'adresse de **loopback** : elle désigne la machine elle-même. Le trafic qui lui est destiné ne quitte jamais l'hôte — il est traité entièrement par l'interface virtuelle `lo`, à l'intérieur de la pile TCP/IP du noyau, sans jamais atteindre la carte réseau physique (Wi-Fi ou Ethernet).

Le test réussit donc tant que la **pile TCP/IP locale** fonctionne, indépendamment de tout réseau externe :

- Aucune carte réseau physique n'est sollicitée : ni Wi-Fi, ni câble, ni passerelle ;
- Aucune résolution DNS ni routage vers l'extérieur n'entre en jeu ;
- Les paquets sont bouclés en interne par le noyau, ce qui explique les temps très faibles observés (RTT moyen $\approx 0,126$ ms).

C'est pour cette raison que le ping du loopback est la **première étape** de la chaîne de diagnostic : s'il échoue, le problème vient de la configuration TCP/IP de la machine elle-même, pas du réseau.

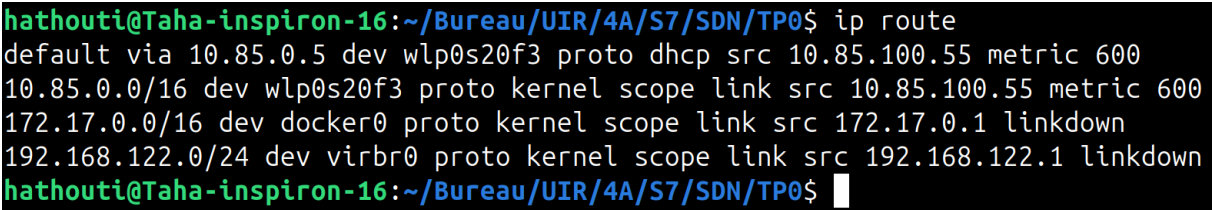
3 Routage et chemin de sortie

Comprendre comment Linux décide où envoyer un paquet.

3.1 Table de routage

Listing 7 – Afficher la table de routage

```
1 ip route
```



The screenshot shows the terminal output of the 'ip route' command. The prompt is 'hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0\$'. The output lists the default route via 10.85.0.5 on interface wlp0s20f3, and two local network routes: 10.85.0.0/16 on wlp0s20f3 and 172.17.0.0/16 on docker0. The output ends with the prompt 'hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0\$'.

FIGURE 6 – Sortie de `ip route`

Élément	Valeur
Passerelle par défaut	10.85.0.5
Interface de sortie	wlp0s20f3 (Wi-Fi)
Réseau directement connecté	10.85.0.0/16 (src 10.85.100.55)

TABLE 6 – Relevé de la table de routage

Interprétez : `default via X.X.X.X dev ens33`.

Cette ligne est la **route par défaut** : elle indique où envoyer tout paquet dont la destination ne correspond à aucun réseau directement connecté (autrement dit, tout ce qui sort du LAN, vers Internet).

- **default** : correspond à 0.0.0.0/0, c'est-à-dire « toutes les destinations non couvertes par une route plus spécifique » ;
- **via X.X.X.X** : l'adresse IP de la **passerelle** (le routeur) à qui remettre ces paquets pour qu'ils soient relayés vers l'extérieur ;
- **dev ens33** : l'**interface de sortie** par laquelle les paquets physiquement partent pour atteindre cette passerelle.

Dans mon cas concret, la ligne est **default via 10.85.0.5 dev wlp0s20f3** : tout le trafic destiné à l'extérieur du réseau 10.85.0.0/16 est envoyé à la passerelle 10.85.0.5 via l'interface Wi-Fi wlp0s20f3. Le `proto dhcp` confirme que cette route a été apprise automatiquement via le DHCP du réseau (celui de l'UIR), et non configurée à la main.

3.2 Quelle route pour une destination ?

Listing 8 – Sélection de route pour une destination

```
1 ip route get 8.8.8.8
2 ip route get 1.1.1.1
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ ip route get 8.8.8.8
8.8.8.8 via 10.85.0.5 dev wlp0s20f3 src 10.85.100.55 uid 1000
cache
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ ip route get 1.1.1.1
1.1.1.1 via 10.85.0.5 dev wlp0s20f3 src 10.85.100.55 uid 1000
cache
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$
```

FIGURE 7 – Sortie de `ip route get 8.8.8.8` et `ip route get 1.1.1.1`

Destination	Interface	Passerelle	Adresse source
8.8.8.8	wlp0s20f3	10.85.0.5	10.85.100.55
1.1.1.1	wlp0s20f3	10.85.0.5	10.85.100.55

TABLE 7 – Relevé de la sélection de route

Question clé pour la suite du module. *Pourquoi `ip route get` sera-t-elle très utile avec une VM qui possède plusieurs cartes réseau ?*

`ip route get` ne se contente pas d'afficher la table de routage : elle demande au noyau la **décision réelle** qu'il prendrait pour une destination donnée. Le noyau applique tout son processus de sélection (route la plus spécifique, métriques, règles de routage) et renvoie la ligne exacte qui sera utilisée : interface de sortie, passerelle et adresse source.

Avec une VM (ou une machine) qui possède **plusieurs cartes réseau**, il y a plusieurs routes possibles et il devient difficile de deviner à l'œil laquelle le noyau va choisir :

- Elle indique par quelle **carte** un paquet sortira réellement (ex. carte NAT vers Internet vs carte host-only vers le réseau interne) ;
- Elle révèle quelle **adresse source** sera utilisée — crucial quand chaque interface a sa propre IP et que le serveur distant doit pouvoir répondre ;
- Elle permet de **vérifier** qu'un trafic donné part bien par la bonne interface avant même d'envoyer quoi que ce soit, ce qui évite de longues sessions de debug ;
- En cas de **multi-homing** ou de routage par règles (*policy routing*), c'est le seul moyen fiable de connaître la route effective sans se fier à la lecture manuelle de la table.

Dans mon cas, les deux destinations 8.8.8.8 et 1.1.1.1 sortent par la même interface Wi-Fi wlp0s20f3 via la passerelle 10.85.0.5, car je n'ai qu'une seule carte réellement active. Sur une VM à plusieurs interfaces, ces valeurs pourraient différer d'une destination à l'autre.

4 Tester la connectivité par couches

Ne commencez jamais par « ping google.com » sans méthode.

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ ping -c 3 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.072 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.074 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.187 ms

--- 127.0.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2065ms
rtt min/avg/max/mdev = 0.072/0.111/0.187/0.053 ms
```

FIGURE 8 – Étape 1 — ping -c 3 127.0.0.1 (pile TCP/IP locale)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=30.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=32.6 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=115 time=49.8 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 30.361/37.583/49.815/8.696 ms
```

FIGURE 9 – Étape 2 — ping -c 3 192.168.11.106 (interface / adresse locale)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ ping -c 3 192.168.11.1
PING 192.168.11.1 (192.168.11.1) 56(84) bytes of data.
64 bytes from 192.168.11.1: icmp_seq=1 ttl=64 time=5.02 ms
64 bytes from 192.168.11.1: icmp_seq=2 ttl=64 time=3.69 ms
64 bytes from 192.168.11.1: icmp_seq=3 ttl=64 time=4.44 ms

--- 192.168.11.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 3.688/4.381/5.016/0.543 ms
```

FIGURE 10 – Étape 3 — ping -c 3 192.168.11.1 (accès au réseau local / passerelle)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ ping -c 3 192.168.11.106
PING 192.168.11.106 (192.168.11.106) 56(84) bytes of data.
64 bytes from 192.168.11.106: icmp_seq=1 ttl=64 time=0.112 ms
64 bytes from 192.168.11.106: icmp_seq=2 ttl=64 time=0.082 ms
64 bytes from 192.168.11.106: icmp_seq=3 ttl=64 time=0.078 ms

--- 192.168.11.106 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2046ms
rtt min/avg/max/mdev = 0.078/0.090/0.112/0.015 ms
```

FIGURE 11 – Étape 4 — ping -c 3 8.8.8.8 (accès Internet sans DNS)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ ping -c 3 google.com
PING google.com (142.251.208.238) 56(84) bytes of data.
64 bytes from ncmrsa-aq-in-f14.1e100.net (142.251.208.238): icmp_seq=1 ttl=114 time=31.2 ms
64 bytes from ncmrsa-aq-in-f14.1e100.net (142.251.208.238): icmp_seq=2 ttl=114 time=32.8 ms
64 bytes from ncmrsa-aq-in-f14.1e100.net (142.251.208.238): icmp_seq=3 ttl=114 time=32.2 ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 31.242/32.087/32.843/0.656 ms
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$
```

FIGURE 12 – Étape 5 — ping -c 3 google.com (Internet + résolution DNS)

Étape	Commande	Ce que vous vérifiez	Résultat
1	ping -c 3 127.0.0.1	Pile TCP/IP locale	☒ OK ☐ KO
2	ping -c 3 192.168.11.106	Interface / adresse locale	☒ OK ☐ KO
3	ping -c 3 192.168.11.1	Accès au réseau local	☒ OK ☐ KO
4	ping -c 3 8.8.8.8	Accès Internet sans DNS	☒ OK ☐ KO
5	ping -c 3 google.com	Internet + résolution DNS	☒ OK ☐ KO

TABLE 8 – Test de connectivité couche par couche

4.1 Mini-diagnostic

On observe : ping passerelle = OK ; ping 8.8.8.8 = OK ; ping google.com = KO.

Cause probable : ☐ Carte réseau ☐ Routage ☒ DNS ☐ CPU

Justification.

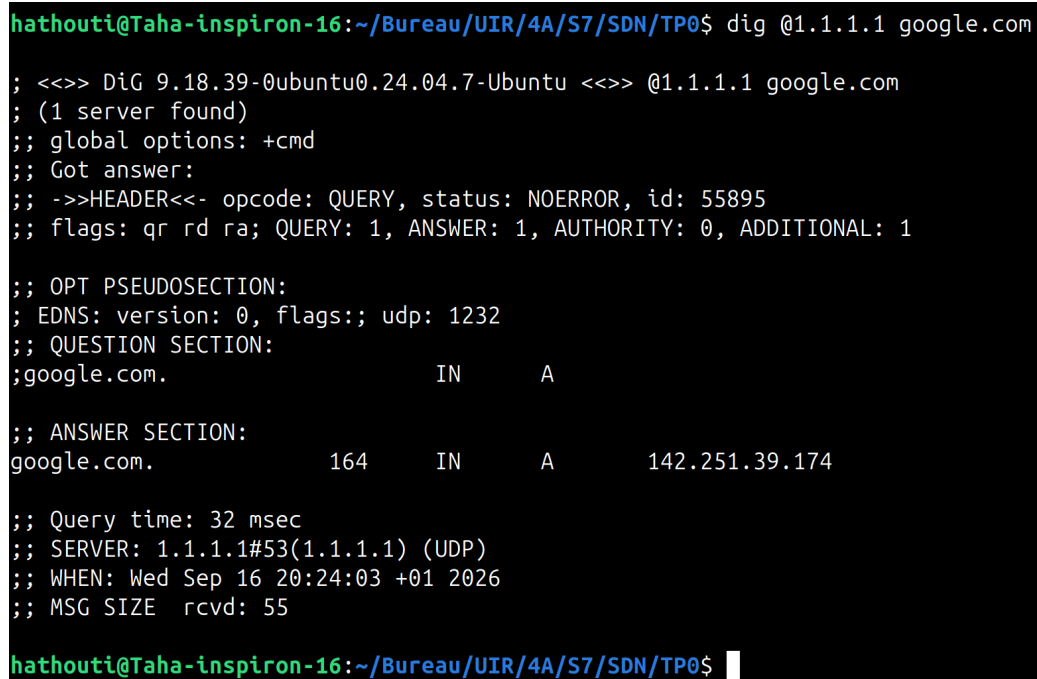
Le ping 8.8.8.8 réussit : la machine atteint donc Internet par **adresse IP**, ce qui prouve que la carte réseau, l'adressage, le routage et la passerelle fonctionnent tous. Seul le ping google.com échoue, c'est-à-dire l'accès par **nom de domaine**. La seule étape qui différencie ces deux tests est la **résolution de nom** : traduire google.com en adresse IP. Comme la connectivité brute est OK mais que la résolution échoue, le problème vient du **DNS** (serveur DNS injoignable, mal configuré, ou /etc/resolv.conf vide), et non de la couche réseau.

5 DNS : résoudre les noms

Différencier un problème de connectivité d'un problème de résolution.

Listing 9 – Diagnostic de la résolution DNS

```
1 resolvectl status
2 cat /etc/resolv.conf
3 dig google.com
4 dig @8.8.8.8 google.com
5 dig @1.1.1.1 google.com
```



```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ dig @1.1.1.1 google.com
; <<>> DiG 9.18.39-0ubuntu0.24.04.7-Ubuntu <<>> @1.1.1.1 google.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 55895
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;google.com.                IN      A
;; ANSWER SECTION:
google.com.                 164     IN      A      142.251.39.174

;; Query time: 32 msec
;; SERVER: 1.1.1.1#53(1.1.1.1) (UDP)
;; WHEN: Wed Sep 16 20:24:03 +01 2026
;; MSG SIZE rcvd: 55

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$
```

FIGURE 13 – Sortie de resolvectl status : serveur DNS 192.168.11.1 sur l'interface Wi-Fi

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ dig @8.8.8.8 google.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.7-Ubuntu <<>> @8.8.8.8 google.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 7194
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;google.com.                IN      A

;; ANSWER SECTION:
google.com.                300     IN      A      142.251.208.238

;; Query time: 56 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
;; WHEN: Wed Sep 16 20:23:36 +01 2026
;; MSG SIZE rcvd: 55

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ █

```

FIGURE 14 – Sortie de cat /etc/resolv.conf : stub resolver systemd-resolved (127.0.0.53)

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ dig google.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.7-Ubuntu <<>> google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 22214
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;google.com.                IN      A

;; ANSWER SECTION:
google.com.                77      IN      A      142.251.208.238

;; Query time: 42 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Wed Sep 16 20:23:04 +01 2026
;; MSG SIZE rcvd: 55

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ █

```

FIGURE 15 – Sortie de dig google.com via le résolveur configuré

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ cat /etc/resolv.conf
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search Home
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$

```

FIGURE 16 – Sortie de dig @8.8.8.8 google.com

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ resolvectl status
Global
    Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
    resolv.conf mode: stub

Link 2 (wlp0s20f3)
    Current Scopes: DNS
    Protocols: +DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
Current DNS Server: 192.168.11.1
    DNS Servers: 192.168.11.1
    DNS Domain: Home

Link 3 (virbr0)
    Current Scopes: none
    Protocols: -DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported

Link 5 (docker0)
    Current Scopes: none
    Protocols: -DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$

```

FIGURE 17 – Sortie de dig @1.1.1.1 google.com

Élément	Valeur
DNS configuré	192.168.11.1 (via le stub systemd-resolved 127.0.0.53)
Une IP retournée pour google.com	142.251.208.238 (status NOERROR)
Résultat avec 8.8.8.8	NOERROR — 142.251.208.238 (Query time 56 ms)
Résultat avec 1.1.1.1	NOERROR — 142.251.39.174 (Query time 32 ms)

TABLE 9 – Relevé DNS

Analyse. Expliquez ce que prouve un `dig` réussi vers 8.8.8.8 alors que la résolution via le DNS configuré échoue.

Un `dig @8.8.8.8 google.com` qui réussit interroge **directement** le résolveur public de Google, en court-circuitant le DNS configuré sur la machine. S’il répond correctement (`status: NOERROR` + une section `ANSWER`), cela prouve deux choses : la **connectivité réseau est fonctionnelle** (les paquets DNS sur le port 53 sortent et reviennent), et le **mécanisme de résolution lui-même fonctionne**. Le problème n’est donc ni la carte réseau, ni le routage, ni Internet : il est **isolé au serveur DNS configuré** sur la machine (celui pointé par `/etc/resolv.conf / resolvectl`). Autrement dit, ce test permet de **localiser précisément la panne** : si `dig @8.8.8.8` marche mais que `dig google.com` (sans serveur explicite) échoue, c’est que le résolveur par défaut est injoignable ou mal configuré, alors que tout le reste de la chaîne est sain.

6 Voisinage local et chemin réseau

Observer ARP/ND et les premiers sauts vers Internet.

6.1 Voisins du LAN

Listing 10 – Table de voisinage (ARP/ND)

```
1 ip neigh
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ ip neigh
192.168.11.1 dev wlp0s20f3 lladdr d8:a0:e8:52:f4:e8 REACHABLE
192.168.11.100 dev wlp0s20f3 lladdr 5e:90:51:76:88:45 REACHABLE
fe80::1 dev wlp0s20f3 lladdr d8:a0:e8:52:f4:e8 router STALE
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$
```

FIGURE 18 – Sortie de `ip neigh`

IP	MAC	Interface	État
192.168.11.1	d8 :a0 :e8 :52 :f4 :e8	wlp0s20f3	REACHABLE
192.168.11.100	5e :90 :51 :76 :88 :45	wlp0s20f3	REACHABLE
fe80 : :1	d8 :a0 :e8 :52 :f4 :e8	wlp0s20f3	STALE (router)

TABLE 10 – Relevé des voisins du LAN

Question. Quel protocole associe une adresse IPv4 locale à une adresse MAC ?

☐ DNS ☐ DHCP ☒ ARP ☐ HTTP

6.2 Tracer le chemin

Listing 11 – Traceroute vers Internet

```
1 sudo apt install traceroute -y
2 traceroute 8.8.8.8
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ which traceroute
/usr/sbin/traceroute
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ traceroute 8.8.8.8
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets
 1 fiberboxip4.home (192.168.11.1)  5.152 ms  5.061 ms  5.031 ms
 2 197.247.192.3 (197.247.192.3)  6.284 ms  6.241 ms  6.214 ms
 3 10.43.12.209 (10.43.12.209)  6.497 ms  6.470 ms  6.443 ms
 4 10.43.12.210 (10.43.12.210)  7.549 ms  7.506 ms  7.479 ms
 5 10.43.250.189 (10.43.250.189)  33.129 ms  33.084 ms  33.058 ms
 6 197.230.70.239 (197.230.70.239)  32.487 ms  142.251.205.34 (142.251.205.34)  30.880 ms  197.230.70.239 (197.230.70.239)  30.610 ms
 7 142.250.232.9 (142.250.232.9)  31.999 ms * *
 8 dns.google (8.8.8.8)  30.592 ms  72.14.233.69 (72.14.233.69)  31.466 ms  216.239.42.99 (216.239.42.99)  30.300 ms
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$
```

FIGURE 19 – Sortie de traceroute 8.8.8.8

Saut	Adresse / nom observé
1	fiberboxip4.home (192.168.11.1) — passerelle locale (ma box)
2	197.247.192.3 — premier routeur du FAI
3	10.43.12.209 — réseau interne du FAI (adresse privée)
4	10.43.12.210 — réseau interne du FAI
5	10.43.250.189 — cœur de réseau du FAI
6	197.230.70.239 / 142.251.205.34 — entrée vers le réseau Google
7	142.250.232.9 — réseau Google (réponses partielles : * *)
8	dns.google (8.8.8.8) — destination finale atteinte

TABLE 11 – Relevé du traceroute vers 8.8.8.8

Interprétation. À quoi correspond généralement le premier saut ? Pourquoi certains sauts peuvent-ils apparaître comme * * * ?

Le **premier saut** correspond toujours à la **passerelle par défaut** du réseau local, c'est-à-dire le routeur qui relie la machine au reste d'Internet. Dans mon cas, c'est `fiberboxip4.home (192.168.11.1)`, autrement dit ma box : c'est le point de sortie obligatoire de tout paquet quittant le LAN, ce qui confirme le relevé de `ip route`. Les sauts affichés en * * * correspondent à des routeurs qui **ne répondent pas** à la sonde de traceroute. Plusieurs raisons possibles :

- Le routeur est configuré pour **ne pas renvoyer de message ICMP Time Exceeded** (par choix de sécurité ou pour limiter la charge) ;
- La réponse arrive **trop tard** et dépasse le délai d'attente de traceroute ;
- Un **pare-feu** en chemin bloque les paquets de sonde ou les réponses ICMP.

Un * ne signifie donc pas que le chemin est coupé : le paquet continue bien vers la destination (ici le saut 8 est atteint normalement), c'est seulement le routeur intermédiaire qui reste silencieux.

7 Ports, services et processus

Faire le lien entre réseau et système.

7.1 Ports en écoute

Listing 12 – Lister les ports en écoute

```
1 ss -lnt
2 sudo ss -lntp
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ ss -lnt
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
LISTEN     0            4096        127.0.0.54:53            0.0.0.0:*
LISTEN     0            4096        127.0.0.1:35541         0.0.0.0:*
LISTEN     0            4096        0.0.0.0:22              0.0.0.0:*
LISTEN     0            151        127.0.0.1:3306          0.0.0.0:*
LISTEN     0            4096        0.0.0.0:32879           0.0.0.0:*
LISTEN     0            4096        127.0.0.1:631           0.0.0.0:*
LISTEN     0            70         127.0.0.1:33060         0.0.0.0:*
LISTEN     0            32         192.168.122.1:53        0.0.0.0:*
LISTEN     0            4096        127.0.0.53%lo:53        0.0.0.0:*
LISTEN     0            5         127.0.0.1:29754         0.0.0.0:*
LISTEN     0            10         0.0.0.0:57621           0.0.0.0:*
LISTEN     0            32         *:21                    *:
LISTEN     0            64         *:23                    *:
LISTEN     0            4096        [::]:22                 [::]:*
LISTEN     0            511        *:80                    *:
LISTEN     0            511        *:443                   *:
LISTEN     0            4096        [::1]:631               [::]:*
```

FIGURE 20 – Sortie de `ss -lnt` (ports TCP en écoute, sans privilèges)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ sudo ss -lntp
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
LISTEN     0            4096        127.0.0.54:53            0.0.0.0:*
users:(("systemd-resolve",pid=178660,fd=17))
LISTEN     0            4096        127.0.0.1:35541         0.0.0.0:*
users:(("containerd",pid=1822,fd=14))
LISTEN     0            4096        0.0.0.0:22              0.0.0.0:*
users:(("systemd",pid=1,fd=434))
LISTEN     0            151        127.0.0.1:3306          0.0.0.0:*
users:(("mysqld",pid=186087,fd=23))
LISTEN     0            4096        0.0.0.0:32879           0.0.0.0:*
users:(("spotify",pid=152837,fd=146))
LISTEN     0            4096        127.0.0.1:631           0.0.0.0:*
users:(("cupsd",pid=1802,fd=8))
LISTEN     0            70         127.0.0.1:33060         0.0.0.0:*
users:(("mysqld",pid=186087,fd=21))
LISTEN     0            32         192.168.122.1:53        0.0.0.0:*
users:(("dnsmasq",pid=2126,fd=6))
LISTEN     0            4096        127.0.0.53%lo:53        0.0.0.0:*
users:(("systemd-resolve",pid=178660,fd=15))
LISTEN     0            5         127.0.0.1:29754         0.0.0.0:*
users:(("vpncagentd",pid=1101,fd=4))
LISTEN     0            10         0.0.0.0:57621           0.0.0.0:*
users:(("spotify",pid=152837,fd=141))
LISTEN     0            32         *:21                    *:
LISTEN     0            64         *:23                    *:
LISTEN     0            4096        [::]:22                 [::]:*
users:(("systemd",pid=1,fd=435))
LISTEN     0            511        *:80                    *:
users:(("apache2",pid=203292,fd=4),("apache2",pid=203291,fd=4),("apache2",pid=203290,fd=4),("apache2",pid=203289,fd=4),("apache2",pid=203288,fd=4),("ap
ache2",pid=203283,fd=4))
LISTEN     0            511        *:443                   *:
users:(("apache2",pid=203292,fd=6),("apache2",pid=203291,fd=6),("apache2",pid=203290,fd=6),("apache2",pid=203289,fd=6),("apache2",pid=203288,fd=6),("ap
```

FIGURE 21 – Sortie de `sudo ss -lntp` (ports TCP en écoute avec processus associés)

Repérez : adresse locale, port, protocole, PID/processus.

La commande `sudo ss -lntp` liste toutes les sockets TCP en état `LISTEN` en révélant, grâce à l'option `-p`, le processus propriétaire et son PID pour chaque port. On distingue les services accessibles uniquement en local (adresse `127.0.0.1` / `127.0.0.53`) de ceux

exposés à tout le réseau (0.0.0.0 ou *). Le tableau ci-dessous fait le lien direct **port** → **processus** → **service** pour les entrées principales.

Port	Adresse locale	Processus (PID)	Service
22	0.0.0.0 / [:]	systemd (1)	SSH (socket activé par systemd)
53	127.0.0.53 / 127.0.0.54	systemd-resolve (178660)	Stub DNS local
53	192.168.122.1	dnsmasq (2126)	DNS/DHCP du pont libvirt virbr0
3306	127.0.0.1	mysqld (186087)	Base de données MySQL/MariaDB
631	127.0.0.1	cupsd (1802)	Serveur d'impression CUPS
21	*	vsftpd (1823)	Serveur FTP (en clair)
23	*	xinetd (1821)	Telnet via xinetd (en clair)
80	*	apache2 (203283..)	Serveur Web HTTP
443	*	apache2 (203283..)	Serveur Web HTTPS
32879 / 57621	0.0.0.0	spotify (152837)	Client Spotify (découverte locale)
29754	127.0.0.1	vpnagentd (1101)	Agent VPN (Cisco AnyConnect)

TABLE 12 – Correspondance port → processus → service (extrait de `sudo ss -lntp`)

On vérifie ainsi qu'un numéro de port ne suffit pas à identifier un service avec certitude : seul le `-p` confirme le processus réel derrière chaque port (par exemple le port 80/443 est ici servi par `apache2`, avec plusieurs processus fils qui partagent le même socket d'écoute).

7.2 Installer un vrai service Web

Listing 13 – Installation et vérification de nginx

```

1 sudo apt update
2 sudo apt install nginx -y
3 systemctl status nginx
4 sudo ss -lntp | grep :80

```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-09-16 20:49:16 +01; 36s ago
     Docs: man:nginx(8)
  Process: 212097 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Process: 212098 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
 Main PID: 212100 (nginx)
    Tasks: 17 (limit: 34674)
   Memory: 11.8M (peak: 13.9M)
      CPU: 64ms
   CGroup: /system.slice/nginx.service
           └─212100 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─212101 "nginx: worker process"
               └─212102 "nginx: worker process"
                 └─212103 "nginx: worker process"
                   └─212104 "nginx: worker process"
                     └─212105 "nginx: worker process"
                       └─212106 "nginx: worker process"
                         └─212107 "nginx: worker process"
                           └─212108 "nginx: worker process"
                             └─212109 "nginx: worker process"
                               └─212111 "nginx: worker process"
                                 └─212112 "nginx: worker process"
                                   └─212113 "nginx: worker process"
                                     └─212114 "nginx: worker process"
                                       └─212115 "nginx: worker process"
                                         └─212116 "nginx: worker process"
                                           └─212117 "nginx: worker process"

sept. 16 20:49:16 Taha-inspiron-16 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
sept. 16 20:49:16 Taha-inspiron-16 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
```

FIGURE 22 – Sortie de `systemctl status nginx` : service actif (Main PID 212100)

Élément	Valeur
Port	80 (*:80 et [::]:80)
Processus	nginx : 1 master (root) + 16 workers (www-data)
PID	212100 (master)
Protocole applicatif	HTTP

TABLE 13 – Relevé du service nginx

7.3 Tester depuis plusieurs points de vue

Listing 14 – Tests HTTP locaux

```
1 curl http://localhost
2 curl http://127.0.0.1
3 curl http://VOTRE_IP
```

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ curl http://localhost
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="UTF-8"><title>TP5 Web Server HTTPS – UIR</title></head>
<body>
  <h1>Serveur Web opérationnel (HTTPS)</h1>
  <p>Namespace : WEB | IP : 192.168.40.2 | Domaine : web.lab.test</p>
  <p>Protocole : HTTPS (TLS) | Certificat auto-signé</p>
  <p>Auteur : HATHOUTI Mohammed Taha – UIR/ESIN 3A Cybersécurité</p>
</body>
</html>
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ curl http://127.0.0.1
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="UTF-8"><title>TP5 Web Server HTTPS – UIR</title></head>
<body>
  <h1>Serveur Web opérationnel (HTTPS)</h1>
  <p>Namespace : WEB | IP : 192.168.40.2 | Domaine : web.lab.test</p>
  <p>Protocole : HTTPS (TLS) | Certificat auto-signé</p>
  <p>Auteur : HATHOUTI Mohammed Taha – UIR/ESIN 3A Cybersécurité</p>
</body>
</html>
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ curl http://192.168.11.106
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="UTF-8"><title>TP5 Web Server HTTPS – UIR</title></head>
<body>
  <h1>Serveur Web opérationnel (HTTPS)</h1>
  <p>Namespace : WEB | IP : 192.168.40.2 | Domaine : web.lab.test</p>
  <p>Protocole : HTTPS (TLS) | Certificat auto-signé</p>
  <p>Auteur : HATHOUTI Mohammed Taha – UIR/ESIN 3A Cybersécurité</p>
</body>
</html>
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$

```

FIGURE 23 – Sortie des trois curl : localhost, 127.0.0.1 et 192.168.11.106

Question. *Ces trois requêtes doivent-elles atteindre le même service ? Expliquez ce qui change entre elles.*

Oui, les trois requêtes atteignent le **même service** (nginx en écoute sur le port 80), car celui-ci écoute sur *:80, c'est-à-dire sur toutes les adresses de la machine. Ce qui change, c'est le **chemin emprunté** pour l'atteindre :

- `http://localhost` : `localhost` est résolu en `127.0.0.1` (via `/etc/hosts`), la requête passe par l'interface **loopback** et ne quitte jamais la machine ;
- `http://127.0.0.1` : identique, mais en visant directement l'adresse de loopback sans passer par la résolution de nom ;
- `http://192.168.11.106` : la requête vise l'adresse de l'interface **réseau** (Wi-Fi), emprunte la pile réseau normale et serait **joignable depuis d'autres machines du LAN**, contrairement aux deux précédentes.

Le service contacté est donc le même, mais seule la troisième requête correspond à un accès réellement exposé sur le réseau ; les deux premières restent strictement internes à la machine.

7.4 Processus et service

Listing 15 – Lien processus / service

```

1 ps aux | grep nginx
2 top
3 sudo systemctl stop nginx

```

```

4 curl http://localhost
5 sudo ss -lntp | grep :80
6 sudo systemctl start nginx

```

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ ps aux | grep nginx
root      212100  0.0  0.0 11172 1796 ?        Ss   20:49   0:00 nginx: master process /usr/sbin/nginx -g daemon on; master_process on;
www-data  212101  0.0  0.0 12952 5140 ?        S    20:49   0:00 nginx: worker process
www-data  212102  0.0  0.0 12952 5140 ?        S    20:49   0:00 nginx: worker process
www-data  212103  0.0  0.0 12952 5140 ?        S    20:49   0:00 nginx: worker process
www-data  212104  0.0  0.0 12952 4776 ?        S    20:49   0:00 nginx: worker process
www-data  212105  0.0  0.0 12952 4776 ?        S    20:49   0:00 nginx: worker process
www-data  212106  0.0  0.0 12952 4776 ?        S    20:49   0:00 nginx: worker process
www-data  212107  0.0  0.0 12952 4776 ?        S    20:49   0:00 nginx: worker process
www-data  212108  0.0  0.0 12952 4776 ?        S    20:49   0:00 nginx: worker process
www-data  212109  0.0  0.0 12952 4776 ?        S    20:49   0:00 nginx: worker process
www-data  212111  0.0  0.0 12952 4776 ?        S    20:49   0:00 nginx: worker process
www-data  212112  0.0  0.0 12952 4776 ?        S    20:49   0:00 nginx: worker process
www-data  212113  0.0  0.0 12952 4776 ?        S    20:49   0:00 nginx: worker process
www-data  212114  0.0  0.0 12952 4776 ?        S    20:49   0:00 nginx: worker process
www-data  212115  0.0  0.0 12952 4776 ?        S    20:49   0:00 nginx: worker process
www-data  212116  0.0  0.0 12952 4776 ?        S    20:49   0:00 nginx: worker process
www-data  212117  0.0  0.0 12952 4712 ?        S    20:49   0:00 nginx: worker process
hathouti  212429  0.0  0.0  9120 2812 pts/1    S+   20:52   0:00 grep --color=auto nginx

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ top
top - 20:52:37 up 9 days, 9:40, 1 user, load average: 0,77, 1,00, 0,95
Tâches: 478 total, 1 en cours, 477 en veille, 0 arrêté, 0 zombie
%Cpu(s):  0,7 ut,  0,4 sy,  0,0 ni, 98,9 id,  0,1 wa,  0,0 hi,  0,0 si,  0,0 st
MiB Mem : 31828,3 total, 4518,4 libr, 18228,0 util, 14769,2 tam/cache
MiB Éch : 4096,0 total, 4094,2 libr, 1,8 util. 13600,3 dispo Mem

  PID UTIL.  PR  NI  VIRT  RES  SHR S  %CPU  %MEM  TEMPS+ COM.
152837 hathouti 20  0 5059844 476848 226468 S  5,6  1,5 10:56.18 spotify
3321 hathouti 20  0 26,5g 620284 502472 S  1,7  1,9 119:36.77 Xorg
3174 hathouti 9 -11 409284 21668 15224 S  1,3  0,1 14:04.07 wireplumber
3562 hathouti 20  0 5073688 467236 156340 S  1,3  1,4 142:36.56 gnome-shell
4344 hathouti 20  0 53,5g 1,8g 673008 S  1,3  5,8 159:41.28 chrome
102259 hathouti 20  0 908864 97208 46408 S  1,3  0,3 2:22.49 gnome-terminal-
186087 mysql 20  0 2310832 397880 36520 S  1,3  1,2 0:50.13 mysqld

```

FIGURE 24 – Arrêt de nginx : le port 80 se libère et curl échoue

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ sudo systemctl stop nginx
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ curl http://localhost
curl: (7) Failed to connect to localhost port 80 after 0 ms: Couldn't connect to server
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ sudo ss -lntp | grep :80
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ sudo systemctl start nginx
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$

```

FIGURE 25 – Sortie de ps aux | grep nginx et top : master + workers nginx

Observation attendue : lorsque nginx est arrêté, le port 80 ne doit plus être en écoute et curl doit échouer.

La commande `ps aux | grep nginx` confirme la structure du service : un processus **master** (PID 212100, lancé par root) qui supervise 16 processus **workers** (utilisateur www-data) se partageant l'écoute du port 80. Après `sudo systemctl stop nginx`, le service n'écoute plus : `sudo ss -lntp | grep :80` ne retourne aucune ligne et `curl http://localhost` échoue avec `curl: (7) Failed to connect... Couldn't connect to server`. Après `sudo systemctl start nginx`, le port 80 est de nouveau en écoute et le service redevient joignable. Cela illustre le lien direct entre un **service** (nginx, géré par systemd) et le **port** qu'il ouvre : arrêter le service ferme le port, ce qui rend l'application injoignable — la distinction clé étant qu'un **processus** est une instance en cours d'exécution, tandis qu'un **service** est un processus géré par systemd (démarrage, arrêt, redémarrage automatique, activation au boot).

8 Logs et ressources système

Un bon diagnostic ne s'arrête pas au réseau.

Listing 16 – Logs et ressources système

```
1 journalctl -u nginx -n 20
2 journalctl -u nginx -f
3 sudo lsof -i :80
4 df -h
5 du -sh /var/log
6 free -h
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ df -h
Sys. de fichiers      Taille Utilisé Dispo Uti% Monté sur
tmpfs                  3,2G   4,0M   3,2G   1% /run
/dev/mapper/vgUbuntu-system 40G    33G   4,7G  88% /
tmpfs                  16G    516M   16G    4% /dev/shm
tmpfs                  5,0M    12K   5,0M    1% /run/lock
efivarfs               374K   249K  121K   68% /sys/firmware/efi/efivars
tmpfs                  16G    220K   16G    1% /run/qemu
/dev/mapper/vgUbuntu-home  53G    36G   15G   71% /home
/dev/nvme0n1p1         196M    96M   101M  49% /boot/efi
tmpfs                  3,2G    204K   3,2G    1% /run/user/1000
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ du -sh /var/log
du: impossible de lire le répertoire '/var/log/sss': Permission non accordée
du: impossible de lire le répertoire '/var/log/private': Permission non accordée
du: impossible de lire le répertoire '/var/log/speech-dispatcher': Permission non accordée
du: impossible de lire le répertoire '/var/log/gdm3': Permission non accordée
du: impossible de lire le répertoire '/var/log/apache2': Permission non accordée
1,2G    /var/log
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ free -h
              total        utilisé        libre       partagé tam/cache    disponible
Mem:           31Gi         17Gi         4,4Gi         5,3Gi         14Gi         13Gi
Échange:        4,0Gi         1,8Mi         4,0Gi
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$
```

FIGURE 26 – Sortie de `journalctl -u nginx -n 20` : historique du service (échec puis démarrage réussi)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ sudo lsof -i :80
COMMAND    PID    USER   FD   TYPE    DEVICE  SIZE/OFF NODE NAME
nginx      212585  root    5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212585  root    6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212586  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212586  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212587  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212587  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212588  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212588  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212589  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212589  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212590  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212590  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212591  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212591  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212592  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212592  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212593  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212593  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212594  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212594  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212596  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212596  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212597  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212597  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212598  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212598  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212599  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212599  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212600  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212600  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212601  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212601  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
nginx      212602  www-data 5u    IPv4    1292470      0t0  TCP *:http (LISTEN)
nginx      212602  www-data 6u    IPv6    1292471      0t0  TCP *:http (LISTEN)
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$
```

FIGURE 27 – Sortie de `journalctl -u nginx -f` : suivi en temps réel des évènements du service

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ journalctl -u nginx -f
sept. 16 20:46:25 Taha-inspiron-16 systemd[1]: nginx.service: Control process exited, code=exited, status=1/FAILURE
sept. 16 20:46:25 Taha-inspiron-16 systemd[1]: nginx.service: Failed with result 'exit-code'.
sept. 16 20:46:25 Taha-inspiron-16 systemd[1]: Failed to start nginx.service - A high performance web server and a reverse proxy server.
sept. 16 20:49:16 Taha-inspiron-16 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
sept. 16 20:49:16 Taha-inspiron-16 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
sept. 16 20:53:26 Taha-inspiron-16 systemd[1]: Stopping nginx.service - A high performance web server and a reverse proxy server...
sept. 16 20:53:26 Taha-inspiron-16 systemd[1]: nginx.service: Deactivated successfully.
sept. 16 20:53:26 Taha-inspiron-16 systemd[1]: Stopped nginx.service - A high performance web server and a reverse proxy server.
sept. 16 20:53:53 Taha-inspiron-16 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
sept. 16 20:53:53 Taha-inspiron-16 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
```

FIGURE 28 – Sortie de `sudo lsof -i :80` : nginx (master + workers) en écoute sur le port 80

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ journalctl -u nginx -n 20
sept. 16 20:46:23 Taha-inspiron-16 nginx[211445]: nginx: [emerg] bind() to [::]:80 failed (98: Address already in use)
sept. 16 20:46:23 Taha-inspiron-16 nginx[211445]: nginx: [emerg] bind() to 0.0.0.0:80 failed (98: Address already in use)
sept. 16 20:46:23 Taha-inspiron-16 nginx[211445]: nginx: [emerg] bind() to [::]:80 failed (98: Address already in use)
sept. 16 20:46:24 Taha-inspiron-16 nginx[211445]: nginx: [emerg] bind() to 0.0.0.0:80 failed (98: Address already in use)
sept. 16 20:46:24 Taha-inspiron-16 nginx[211445]: nginx: [emerg] bind() to [::]:80 failed (98: Address already in use)
sept. 16 20:46:24 Taha-inspiron-16 nginx[211445]: nginx: [emerg] bind() to 0.0.0.0:80 failed (98: Address already in use)
sept. 16 20:46:24 Taha-inspiron-16 nginx[211445]: nginx: [emerg] bind() to [::]:80 failed (98: Address already in use)
sept. 16 20:46:25 Taha-inspiron-16 nginx[211445]: nginx: [emerg] bind() to 0.0.0.0:80 failed (98: Address already in use)
sept. 16 20:46:25 Taha-inspiron-16 nginx[211445]: nginx: [emerg] bind() to [::]:80 failed (98: Address already in use)
sept. 16 20:46:25 Taha-inspiron-16 nginx[211445]: nginx: [emerg] still could not bind()
sept. 16 20:46:25 Taha-inspiron-16 systemd[1]: nginx.service: Control process exited, code=exited, status=1/FAILURE
sept. 16 20:46:25 Taha-inspiron-16 systemd[1]: nginx.service: Failed with result 'exit-code'.
sept. 16 20:46:25 Taha-inspiron-16 systemd[1]: Failed to start nginx.service - A high performance web server and a reverse proxy server.
sept. 16 20:49:16 Taha-inspiron-16 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
sept. 16 20:49:16 Taha-inspiron-16 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
sept. 16 20:53:26 Taha-inspiron-16 systemd[1]: Stopping nginx.service - A high performance web server and a reverse proxy server...
sept. 16 20:53:26 Taha-inspiron-16 systemd[1]: nginx.service: Deactivated successfully.
sept. 16 20:53:26 Taha-inspiron-16 systemd[1]: Stopped nginx.service - A high performance web server and a reverse proxy server.
sept. 16 20:53:53 Taha-inspiron-16 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
sept. 16 20:53:53 Taha-inspiron-16 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$
```

FIGURE 29 – Sortie de `df -h`, du `du -sh /var/log` et `free -h` : disque, logs et mémoire

Commande	Ce qu'elle permet de vérifier
<code>journalctl -u nginx</code>	Consulter les journaux d'un service précis : ici l'échec initial (<code>bind()</code> ... <code>Address already in use</code>) puis les démarrages/arrêts réussis. L'option <code>-n 20</code> limite aux 20 dernières lignes, <code>-f</code> suit les nouveaux évènements en temps réel.
<code>lsof -i :80</code>	Identifier quel(s) processus utilise(nt) un port réseau donné : ici nginx (PID 212585 master + workers www-data) en écoute sur <code>*:http</code> en IPv4 et IPv6.
<code>df -h</code>	Afficher l'espace disque par système de fichiers monté : la racine <code>/</code> est utilisée à 88% (33G/40G), <code>/home</code> à 71% (36G/53G).
<code>du -sh /var/log</code>	Mesurer la taille occupée par les journaux système : ici 1,2G pour <code>/var/log</code> (certains sous-dossiers nécessitent les droits root).
<code>free -h</code>	Afficher l'utilisation de la mémoire : 31 Gio de RAM (17 Gio utilisés, 13 Gio disponibles) et 4 Gio de swap quasi inutilisé (1,8 Mio).

TABLE 14 – Rôle des commandes de logs et ressources

9 Challenge final : la panne mystérieuse

Vous avez 15 minutes pour produire un diagnostic argumenté.

Scénario : Un utilisateur signale : « Le serveur Web ne fonctionne plus. » Votre enseignant peut provoquer une panne : nginx arrêté, interface indisponible, DNS cassé, route manquante, port fermé ou mauvaise configuration. Vous ne savez pas laquelle.

9.1 Méthode obligatoire

Ordre imposé : Système → Interface UP → Adresse IP → Route → Passerelle → Internet sans DNS → DNS → Port 80 → Service nginx → Application via `curl`.

9.2 Fiche d'incident à compléter

Point de contrôle	Résultat / preuve
Symptôme observé	<code>curl http://localhost</code> échoue : Failed to connect to localhost port 80 (Connection refused)
Interface réseau	wlp0s20f3 (Wi-Fi) UP, LOWER_UP
Adresse IP	192.168.11.106/24
Passerelle	192.168.11.1 (ping OK)
Internet	☒ OK ☐ KO
DNS	☒ OK ☐ KO
Port 80	☐ Ouvert ☒ Fermé
nginx	☐ Actif ☒ Inactif
Cause trouvée	Le service nginx est arrêté (inactive (dead)) : aucun processus n'écoute sur le port 80
Commande décisive	<code>systemctl status nginx</code> (confirmé par <code>sudo ss -lntp grep :80</code> qui ne retourne rien)
Correction proposée	<code>sudo systemctl start nginx</code>

TABLE 15 – Fiche d'incident du challenge

Raisonnement. En remontant la chaîne de diagnostic dans l'ordre imposé, chaque maillon est validé : le système répond, l'interface `wlp0s20f3` est UP, l'adresse IP et la route par défaut sont présentes, la passerelle répond au ping, l'accès Internet par IP (8.8.8.8) fonctionne et la résolution DNS aussi (`google.com` répond). La rupture n'apparaît qu'à la toute fin de la chaîne : le port 80 n'est plus en écoute (`ss -lntp | grep :80` ne retourne rien) et `systemctl status nginx` indique **inactive (dead)**. La panne est donc isolée au niveau **applicatif** (le service nginx), et non au niveau réseau : tout le reste de la chaîne étant sain, seule la relance du service (`sudo systemctl start nginx`) est nécessaire, ce que confirme le retour de la page servie par `curl`. Cette approche descendante évite de chercher au mauvais endroit : plutôt que de suspecter le réseau, on prouve couche par couche ce qui fonctionne pour ne retenir que la cause réelle.

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ sudo systemctl stop nginx
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ systemctl status nginx
Onginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: inactive (dead) since Wed 2026-09-16 21:08:27 +01; 6s ago
     Duration: 14min 34.359s
       Docs: man:nginx(8)
    Process: 212582 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 212583 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 213695 ExecStop=/sbin/start-stop-daemon --quiet --stop --retry QUIT/5 --pidfile /run/nginx.pid (code=exited, status=0/SUCCESS)
   Main PID: 212585 (code=exited, status=0/SUCCESS)
      CPU: 110ms

sept. 16 20:53:53 Taha-inspiron-16 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
sept. 16 20:53:53 Taha-inspiron-16 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
sept. 16 21:08:27 Taha-inspiron-16 systemd[1]: Stopping nginx.service - A high performance web server and a reverse proxy server...
sept. 16 21:08:27 Taha-inspiron-16 systemd[1]: nginx.service: Deactivated successfully.
sept. 16 21:08:27 Taha-inspiron-16 systemd[1]: Stopped nginx.service - A high performance web server and a reverse proxy server.
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ sudo ss -lntp | grep :80
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ curl http://localhost
curl: (7) Failed to connect to localhost port 80 after 0 ms: Couldn't connect to server
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ sudo systemctl start nginx
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$ curl http://localhost
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="UTF-8"><title>TP5 Web Server HTTPS – UIR</title></head>
<body>
  <h1>Serveur Web opérationnel (HTTPS)</h1>
  <p>Namespace : WEB | IP : 192.168.40.2 | Domaine : web.lab.test</p>
  <p>Protocole : HTTPS (TLS) | Certificat auto-signé</p>
  <p>Auteur : HATHOUTI Mohammed Taha – UIR/ES1N 3A Cybersécurité</p>
</body>
</html>
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP0$
```

FIGURE 30 – Challenge : panne simulée (nginx arrêté, port 80 fermé, curl en échec) puis correction (start et page à nouveau servie)

10 Synthèse et transition vers Network Virtualization

Ce que vous devez maîtriser avant le TP VMware.

Chaîne de diagnostic à retenir :

Système → Interface → IP → Route → Passerelle → Internet → DNS → Port
→ Service → Application

10.1 Commandes indispensables

Besoin	Commande
Identifier la machine	hostname / uname -a
Interfaces	ip link
Adresses IP	ip addr
Routes	ip route
Choix de route	ip route get DESTINATION
Voisins	ip neigh
Connectivité	ping
Chemin	traceroute
DNS	resolvectl / dig
Ports	ss -lntp
HTTP	curl
Processus	ps / top
Services	systemctl
Logs	journalctl
Processus sur un port	lsof
RAM	free -h
Disque	df -h / du -sh

TABLE 16 – Aide-mémoire des commandes indispensables

10.2 Questions finales

10.2.1 Quelle différence entre ip addr et ip route ?

`ip addr` affiche (ou configure) les **adresses IP** portées par chaque interface : quelle IPv4/IPv6, quel préfixe, quelle adresse MAC, sur quelle carte. `ip route` affiche (ou configure) la **table de routage** : par où la machine envoie les paquets selon leur destination, notamment via la passerelle par défaut. En résumé, `ip addr` répond à « quelle est mon identité réseau ? » tandis que `ip route` répond à « par où sortent mes paquets ? ».

10.2.2 Pourquoi 8.8.8.8 peut-il répondre alors que google.com échoue ?

8.8.8.8 est une **adresse IP directe** : la joindre ne teste que la connectivité réseau brute (carte, routage, passerelle, Internet). `google.com` est un **nom de domaine** : il faut d'abord le traduire en adresse IP via une requête DNS avant de pouvoir l'atteindre. Si 8.8.8.8 répond mais que `google.com` échoue, la connectivité est donc saine et le

problème est **isolé au DNS** (serveur injoignable, mal configuré ou `/etc/resolv.conf` vide).

10.2.3 À quoi sert la passerelle par défaut ?

La passerelle par défaut est le **routeur** vers lequel la machine envoie tout paquet destiné à un réseau **autre que le sien**. Sans elle, la machine ne peut communiquer qu'avec les hôtes de son propre sous-réseau local. C'est donc le **point de sortie obligatoire** vers les autres réseaux et vers Internet, correspondant à la ligne `default via ...` de la table de routage.

10.2.4 Quelle commande permet de savoir quel processus écoute sur TCP/80 ?

La commande `sudo ss -lntp | grep :80` liste la socket en écoute sur le port 80 avec le processus et son PID (grâce à l'option `-p`). La commande `sudo lsof -i :80` donne le même résultat sous une autre forme (commande, PID, utilisateur, type de socket). Le `sudo` est nécessaire pour voir les processus appartenant à d'autres utilisateurs.

10.2.5 Quelle différence entre un processus et un service Linux ?

Un **processus** est une instance d'un programme en cours d'exécution, identifiée par un PID, créée et détruite à la demande. Un **service** (ou daemon) est un processus qui tourne en arrière-plan et qui est **géré par systemd** : démarrage, arrêt, redémarrage automatique en cas d'échec, activation au boot et supervision. Par exemple, nginx est un **service** qui, une fois lancé par systemd, crée plusieurs **processus** (un master et ses workers) partageant l'écoute du port 80.

10.2.6 Pourquoi `ip route get` devient-il important avec plusieurs interfaces réseau ?

Avec plusieurs cartes réseau, plusieurs routes sont possibles et il devient difficile de deviner laquelle le noyau va réellement utiliser. `ip route get DESTINATION` demande au noyau la **décision effective** pour une destination donnée : interface de sortie, passerelle et adresse source retenues. C'est donc le seul moyen fiable de savoir par quelle carte un trafic sortira réellement, ce qui est crucial en **multi-homing** ou en routage par règles (*policy routing*).

Livrable étudiant

- ☒ Capture de `ip addr` ;
- ☒ Capture de `ip route` ;
- ☒ Résultat de `ip route get 8.8.8.8` ;
- ☒ Capture de `ip neigh` ;
- ☒ Capture du `traceroute` ;
- ☒ DNS identifié ;
- ☒ Capture de `ss -lntp` avec nginx ;

- ☒ Résultat de `curl` ;
- ☒ État `systemctl` de `nginx` ;
- ☒ Fiche d'incident du challenge ;
- ☒ Réponses aux six questions finales.